

Punto 3.3 – Proyecciones con Select y tipos anónimos

Práctica

Ejercicio: Añadir proyecciones al repositorio de órdenes del proyecto AceriaData. Crear métodos que devuelvan tipos anónimos y DTOs. Crear un caso de uso que ejecute estas proyecciones, analice el SQL generado y compare el rendimiento de proyectar versus cargar entidades completas.

Contexto del proyecto: En el punto 3.2 se añadieron consultas básicas con Where, OrderBy y ThenBy. En este punto se añaden proyecciones con Select, incluyendo tipos anónimos y DTOs. Estas proyecciones se usarán en el punto 3.4 para las proyecciones a DTOs y en el punto 3.5 para las consultas de agregación.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO OrdenResumenDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenResumenDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenResumenDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres de los DTOs.

Línea 3: public class OrdenResumenDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → propiedad que almacena el número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → propiedad que almacena el cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → propiedad que almacena el estado.

Línea 8: public DateTime FechaCreacion { get; set; } → propiedad que almacena la fecha de creación.

Error común: si el DTO tiene propiedades que no se proyectan, EF Core no las incluye en el SELECT. Se deben proyectar todas las propiedades que se van a usar.

Paso 3: Crear el DTO OrdenConTotalesDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConTotalesDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConTotalesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres de los DTOs.

Línea 3: public class OrdenConTotalesDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → propiedad que almacena el número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → propiedad que almacena el cliente.

Línea 7: public int TotalPlanchas { get; set; } → propiedad que almacena el total de planchas.

Línea 8: public decimal PesoTotal { get; set; } → propiedad que almacena el peso total.

Error común: si el DTO usa int para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar decimal para los pesos.

| Paso 4: Añadir los métodos de proyección a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: `IQueryable ObtenerQueryable();` → método que devuelve `IQueryable`.

Línea 11: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → método que obtiene por número.

Línea 12: `List ObtenerPendientesPorCliente(string cliente);` → método que obtiene las pendientes de un cliente.

Línea 13: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → método que obtiene por estado ordenadas.

Línea 14: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene por rango de fechas.

Línea 15: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene por cliente ordenadas.

Línea 16: `List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);` → método que obtiene por cliente y rango de fechas.

Línea 18: `List ObtenerClientesUnicos();` → método que obtiene los clientes únicos.

Línea 19: `List ObtenerResumenes();` → método que obtiene los resúmenes de todas las órdenes.

Línea 20: `List ObtenerResumenesPorEstado(string estado);` → método que obtiene los resúmenes por estado.

Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales de planchas y peso.

Línea 23: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 24: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 5: Implementar los métodos de proyección en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories.OrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context)
    {
        _context = context;
    }

    public OrdenFabricacion? ObtenerPorId(int id)
    {
        return _context.OrdenesFabricacion.Find(id);
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.ToList();
    }

    public IQueryable<OrdenFabricacion> ObtenerQueryable()
    {
        return _context.OrdenesFabricacion.AsQueryable();
    }
}
```

```

    }

    public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden)
    {
        return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == estado)
            .OrderByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente)
            .OrderBy(o => o.Cliente)
            .ThenByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde,
DateTime hasta)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
            .OrderBy(o => o.Estado)
            .ThenByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<string> ObtenerClientesUnicos()
    {
        return _context.OrdenesFabricacion
            .Select(o => o.Cliente)
            .Distinct()
            .OrderBy(c => c)
            .ToList();
    }

    public List<OrdenResumenDto> ObtenerResumenes()
    {
        return _context.OrdenesFabricacion
            .OrderBy(o => o.FechaCreacion)
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .ToList();
    }
}

```

```

public List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenConTotalesDto> ObtenerOrdenesConTotales()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConTotalesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            TotalPlanchas = o.Planchas.Count(),
            PesoTotal = o.Planchas.Sum(p => p.Peso)
        })
        .ToList();
}

public void Agregar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Add(orden);
}

public void Eliminar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Remove(orden);
}
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 3: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 6: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 8: public class OrdenRepositorio : IOrdenRepositorio → declara la implementación.

Línea 10: private readonly AceriaDbContext _context; → campo del DbContext.

Línea 12: public OrdenRepositorio(AceriaDbContext context) → constructor.

Línea 14: _context = context; → asigna el parámetro al campo.

Línea 17: public OrdenFabricacion? ObtenerPorId(int id) → método que obtiene por Id.

Línea 19: return _context.OrdenesFabricacion.Find(id); → busca por clave primaria.

Línea 22: public List ObtenerTodas() → método que obtiene todas.

Línea 24: return _context.OrdenesFabricacion.ToList(); → materializa la consulta.

Línea 27: public IQueryable ObtenerQueryable() → método que devuelve IQueryable.

Línea 29: return _context.OrdenesFabricacion.AsQueryable(); → devuelve la consulta sin materializar.

Línea 32: public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden) → método que obtiene por número.

Línea 34: `return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → filtra por número.

Línea 37: `public List ObtenerPendientesPorCliente(string cliente)` → método que obtiene las pendientes de un cliente.

Línea 39: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 40: `.Where(o => o.Cliente == cliente && o.Estado == "Pendiente")` → filtra por cliente y estado.

Línea 41: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 42: `.ToList();` → materializa la consulta.

Línea 45: `public List ObtenerPorEstadoOrdenadasPorFecha(string estado)` → método que obtiene por estado.

Línea 47: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 48: `.Where(o => o.Estado == estado)` → filtra por estado.

Línea 49: `.OrderByDescending(o => o.FechaCreacion)` → ordena por fecha descendente.

Línea 50: `.ToList();` → materializa la consulta.

Línea 53: `public List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)` → método que obtiene por rango de fechas.

Línea 55: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 56: `.Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)` → filtra por rango.

Línea 57: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 58: `.ToList();` → materializa la consulta.

Línea 61: `public List ObtenerPorClienteOrdenadas(string cliente)` → método que obtiene por cliente.

Línea 63: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 64: `.Where(o => o.Cliente == cliente)` → filtra por cliente.

Línea 65: `.OrderBy(o => o.Cliente)` → ordena por cliente.

Línea 66: `.ThenByDescending(o => o.FechaCreacion)` → añade criterio secundario.

Línea 67: `.ToList();` → materializa la consulta.

Línea 70: `public List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta)` → método que combina cliente y rango.

Línea 72: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 73: `.Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)` → filtra por cliente y rango.

Línea 74: `.OrderBy(o => o.Estado)` → ordena por estado.

Línea 75: `.ThenByDescending(o => o.FechaCreacion)` → añade criterio secundario.

Línea 76: `.ToList();` → materializa la consulta.

Línea 79: `public List ObtenerClientesUnicos()` → método que obtiene los clientes únicos.

Línea 81: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 82: `.Select(o => o.Cliente)` → proyecta a la propiedad Cliente.

Línea 83: `.Distinct()` → elimina duplicados.

Línea 84: `.OrderBy(c => c)` → ordena alfabéticamente.

Línea 85: .ToList(); → materializa la consulta.

Línea 88: public List ObtenerResumenes() → método que obtiene los resúmenes.

Línea 90: return _context.OrdenesFabricacion → inicia la consulta.

Línea 91: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 92: .Select(o => new OrdenResumenDto → proyecta a OrdenResumenDto.

Línea 93: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 94: Cliente = o.Cliente, → asigna el cliente.

Línea 95: Estado = o.Estado, → asigna el estado.

Línea 96: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 97: }) → cierra la proyección.

Línea 98: .ToList(); → materializa la consulta.

Línea 101: public List ObtenerResumenesPorEstado(string estado) → método que obtiene los resúmenes por estado.

Línea 103: return _context.OrdenesFabricacion → inicia la consulta.

Línea 104: .Where(o => o.Estado == estado) → filtra por estado.

Línea 105: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 106: .Select(o => new OrdenResumenDto → proyecta a OrdenResumenDto.

Línea 107: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 108: Cliente = o.Cliente, → asigna el cliente.

Línea 109: Estado = o.Estado, → asigna el estado.

Línea 110: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 111: }) → cierra la proyección.

Línea 112: .ToList(); → materializa la consulta.

Línea 115: public List ObtenerOrdenesConTotales() → método que obtiene las órdenes con totales.

Línea 117: return _context.OrdenesFabricacion → inicia la consulta.

Línea 118: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 119: .Select(o => new OrdenConTotalesDto → proyecta a OrdenConTotalesDto.

Línea 120: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 121: Cliente = o.Cliente, → asigna el cliente.

Línea 122: TotalPlanchas = o.Planchas.Count(), → calcula el total de planchas.

Línea 123: PesoTotal = o.Planchas.Sum(p => p.Peso) → calcula el peso total.

Línea 124: }) → cierra la proyección.

Línea 125: .ToList(); → materializa la consulta.

Línea 128: public void Agregar(OrdenFabricacion orden) → método que agrega una orden.

Línea 130: _context.OrdenesFabricacion.Add(orden); → registra la entidad.

Línea 133: public void Eliminar(OrdenFabricacion orden) → método que elimina una orden.

Línea 135: _context.OrdenesFabricacion.Remove(orden); → marca la entidad para eliminar.

Error común: si se proyecta a un DTO con una propiedad que no existe en la entidad, el compilador lanza un error. Se deben proyectar solo propiedades que existen.

| Paso 6: Crear el caso de uso de proyecciones

Crear el archivo src/AceriaData.Application/UseCases/ProyeccionesUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ProyeccionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ProyeccionesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES CON SELECT ===");

        DemostrarClientesUnicos();
        DemostrarResumenes();
        DemostrarResumenesPorEstado();
        DemostrarOrdenesConTotales();
        DemostrarSqlProyeccion();
    }

    private void DemostrarClientesUnicos()
    {
        Console.WriteLine("\n--- Clientes únicos ---");

        var clientes = _unidad.Ordenes.ObtenerClientesUnicos();
        foreach (var cliente in clientes)
        {
            Console.WriteLine($" {cliente}");
        }
    }

    private void DemostrarResumenes()
    {
        Console.WriteLine("\n--- Resúmenes de órdenes ---");

        var resumenes = _unidad.Ordenes.ObtenerResumenes();
        foreach (var resumen in resumenes)
        {
            Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarResumenesPorEstado()
    {
        Console.WriteLine("\n--- Resúmenes de órdenes pendientes ---");

        var resumenes = _unidad.Ordenes.ObtenerResumenesPorEstado("Pendiente");
        foreach (var resumen in resumenes)
        {
            Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarOrdenesConTotales()
    {
        Console.WriteLine("\n--- Órdenes con totales de planchas y peso ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotales();
        foreach (var orden in ordenes)
    }
}
```

```

        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas:
{orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg");
        }
    }

    private void DemostrarSqlProyeccion()
    {
        Console.WriteLine("\n--- SQL generado por una proyección ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .Select(o => new
            {
                o.NumeroOrden,
                o.Cliente,
                o.Estado
            });

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class ProyeccionesUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public ProyeccionesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== PROYECCIONES CON SELECT ==="); → muestra la cabecera.

Línea 18: DemostrarClientesUnicos(); → llama al método de clientes únicos.

Línea 19: DemostrarResumenes(); → llama al método de resúmenes.

Línea 20: DemostrarResumenesPorEstado(); → llama al método de resúmenes por estado.

Línea 21: DemostrarOrdenesConTotales(); → llama al método de órdenes con totales.

Línea 22: DemostrarSqlProyeccion(); → llama al método de SQL de proyección.

Línea 25: private void DemostrarClientesUnicos() → declara el método.

Línea 27: Console.WriteLine("\n--- Clientes únicos ---"); → muestra la cabecera.

Línea 29: var clientes = _unidad.Ordenes.ObtenerClientesUnicos(); → llama al método del repositorio.

Línea 30: foreach (var cliente in clientes) → itera sobre los clientes.

Línea 32: Console.WriteLine(\$" {cliente}"); → muestra el cliente.

Línea 35: private void DemostrarResumenes() → declara el método.

Línea 37: Console.WriteLine("\n--- Resúmenes de órdenes ---"); → muestra la cabecera.

Línea 39: var resumenes = _unidad.Ordenes.ObtenerResumenes(); → llama al método del repositorio.

Línea 40: foreach (var resumen in resumenes) → itera sobre los resúmenes.

Línea 42: Console.WriteLine(\$" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Línea 45: `private void DemostrarResumenesPorEstado()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Resúmenes de órdenes pendientes ---");` → muestra la cabecera.

Línea 49: `var resumenes = _unidad.Ordenes.ObtenerResumenesPorEstado("Pendiente");` → llama al método del repositorio.

Línea 50: `foreach (var resumen in resumenes)` → itera sobre los resúmenes.

Línea 52: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 55: `private void DemostrarOrdenesConTotales()` → declara el método.

Línea 57: `Console.WriteLine("\n--- Órdenes con totales de planchas y peso ---");` → muestra la cabecera.

Línea 59: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotales();` → llama al método del repositorio.

Línea 60: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 62: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas: {orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg");` → muestra los datos.

Línea 65: `private void DemostrarSqlProyeccion()` → declara el método.

Línea 67: `Console.WriteLine("\n--- SQL generado por una proyección ---");` → muestra la cabecera.

Línea 69: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 70: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 71: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 72: `.Select(o => new` → proyecta a un tipo anónimo.

Línea 73: `o.NumeroOrden,` → incluye el número de orden.

Línea 74: `o.Cliente,` → incluye el cliente.

Línea 75: `o.Estado` → incluye el estado.

Línea 76: `});` → cierra la proyección.

Línea 78: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 79: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ProyeccionesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas de prueba:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };

    context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 11: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes.

Línea 12: `context.SaveChanges();` → inserta las órdenes.

Línea 14: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 15: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 16: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 18: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 19: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 10: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada proyección. Se observan los clientes únicos, los resúmenes de órdenes, los resúmenes por estado, las órdenes con totales de planchas y peso, y el SQL generado por la proyección.

Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PROYECCIONES CON SELECT ===
```

```
--- Clientes únicos ---
```

```
  Constructora del Norte
  Constructora del Sur
```

```
--- Resúmenes de órdenes ---
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024
OF-2024-0003 | Constructora del Norte | EnProceso | 10/03/2024
```

```
--- Resúmenes de órdenes pendientes ---
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024
```

```
--- Órdenes con totales de planchas y peso ---
```

```
OF-2024-0001 | Constructora del Norte | Planchas: 2 | Peso total: 651.3 kg
OF-2024-0002 | Constructora del Sur | Planchas: 1 | Peso total: 125.6 kg
OF-2024-0003 | Constructora del Norte | Planchas: 0 | Peso total: 0 kg
```

```
--- SQL generado por una proyección ---
```

```
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente'
ORDER BY [o].[FechaCreacion]
```

La primera sección muestra los clientes únicos. La segunda sección muestra los resúmenes de todas las órdenes. La tercera sección muestra los resúmenes de las órdenes pendientes. La cuarta sección muestra las órdenes con totales de planchas y peso. La quinta sección muestra el SQL generado por la proyección.

Observaciones: el SQL generado solo incluye las columnas proyectadas, no todas las columnas de la entidad. Esto reduce el volumen de datos transferidos.

Paso 12: Diagnosticar un error común

Modificar el método `ObtenerResumenes` para proyectar solo algunas propiedades y acceder a una propiedad no proyectada:

```
csharp
public List<OrdenResumenDto> ObtenerResumenes()
{
    return _context.OrdenesFabricacion
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente
        })
        .ToList();
}
```

```

    })
    .ToList();
}

```

Resultado esperado: el compilador lanza un error porque las propiedades Estado y FechaCreacion no se han asignado en la proyección. El DTO requiere que todas sus propiedades se asignen.

Solución: proyectar todas las propiedades del DTO o marcar las no proyectadas como anulables.

Errores comunes del ejercicio Error Causa Solución Propiedad no proyectada Se accede a una propiedad que no está en la proyección Proyectar todas las propiedades necesarias Tipo anónimo fuera de ámbito Se intenta usar un tipo anónimo fuera del método Proyectar a un DTO con nombre Función no traducida Se usa una función que EF Core no puede traducir Usar funciones traducibles Proyección antes del filtro Se proyecta antes de filtrar Filtrar y ordenar antes de proyectar Colección de navegación sin ToList Se proyecta una colección sin materializarla Añadir ToList dentro de la proyección Distinct no aplicado Se olvidó el operador Distinct Añadir .Distinct() antes de ToList

Retos resueltos: Proyección de órdenes con cliente y número de planchas

Reto: Crear un método en el repositorio que proyecte las órdenes a un DTO con el número de orden, el cliente, el estado y el número de planchas. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO OrdenConNumeroPlanchasDto:

```

namespace AceriaData.Application.Dtos;

public class OrdenConNumeroPlanchasDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public int NumeroPlanchas { get; set; }
}

```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenConNumeroPlanchasDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → propiedad que almacena el número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → propiedad que almacena el cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → propiedad que almacena el estado.

Línea 8: public int NumeroPlanchas { get; set; } → propiedad que almacena el número de planchas.

Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```

List<OrdenConNumeroPlanchasDto> ObtenerOrdenesConNumeroPlanchas();

```

Línea 1: List ObtenerOrdenesConNumeroPlanchas(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```

public List<OrdenConNumeroPlanchasDto> ObtenerOrdenesConNumeroPlanchas()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConNumeroPlanchasDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            NumeroPlanchas = o.Planchas.Count()
        })
        .ToList();
}

```

Línea 1: public List ObtenerOrdenesConNumeroPlanchas() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 5: .Select(o => new OrdenConNumeroPlanchasDto → proyecta al DTO.

Línea 6: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 7: Cliente = o.Cliente, → asigna el cliente.

Línea 8: Estado = o.Estado, → asigna el estado.

Línea 9: NumeroPlanchas = o.Planchas.Count() → calcula el número de planchas.

Línea 10: }) → cierra la proyección.

Línea 11: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```

private void DemostrarOrdenesConNumeroPlanchas()
{
    Console.WriteLine("\n--- Órdenes con número de planchas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConNumeroPlanchas();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas:
{orden.NumeroPlanchas}");
    }
}

```

Línea 1: private void DemostrarOrdenesConNumeroPlanchas() → declara el método.

Línea 3: Console.WriteLine("\n--- Órdenes con número de planchas ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerOrdenesConNumeroPlanchas(); → llama al método del repositorio.

Línea 6: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 8: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas: {orden.NumeroPlanchas}"); → muestra los datos.

Paso 5: Llamar al método desde Ejecutar:

```

DemostrarOrdenesConNumeroPlanchas();

```

Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran con su número de planchas.

Resultado esperado: las órdenes se muestran con su número de orden, cliente, estado y número de planchas.

| Analogía final

Las proyecciones con Select en una acería son como los resúmenes que el jefe de planta pide al archivo central. En lugar de recibir la carpeta completa de cada orden con todos sus documentos, el jefe pide solo los datos que necesita: el número de orden, el cliente y el estado. El archivo central prepara un resumen con esos datos y lo entrega. El jefe no necesita los documentos completos para tomar decisiones: solo los datos clave. Proyectar a un tipo anónimo es como pedir un resumen informal: se usa en el momento y no se archiva. Proyectar a un DTO es como pedir un formulario estandarizado: tiene nombre, se puede archivar y se puede usar en otros departamentos. Proyectar con Distinct es como pedir la lista de clientes únicos: sin duplicados. Proyectar con funciones de agregación es como pedir el total de planchas y el peso total: valores calculados en el archivo central. Así funcionan las proyecciones en EF Core: se seleccionan solo los datos necesarios, se reduce el volumen de información y se mejora el rendimiento.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs OrdenResumenDto y OrdenConTotalesDto.

Añadido los métodos de proyección a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso ProyeccionesUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas.

Ejecutado las demostraciones de clientes únicos, resúmenes, resúmenes por estado y órdenes con totales.

Analizado el SQL generado por una proyección.

Diagnosticado el error de acceder a una propiedad no proyectada.

Creado el DTO OrdenConNumeroPlanchasDto y su método.

| Conexión con el siguiente punto

En este punto se han añadido proyecciones con Select al proyecto AceriaData, incluyendo tipos anónimos, DTOs con nombre, proyecciones de propiedades individuales, Distinct y funciones de agregación. Se ha analizado el SQL generado por las proyecciones y se ha comparado el rendimiento de proyectar versus cargar entidades completas. En el siguiente punto se estudiarán las proyecciones a DTOs en detalle, incluyendo el uso de constructores, la proyección de colecciones de navegación y la proyección con SelectMany.
